SpecRover: Code Intent Extraction via LLMs

Haifeng Ruan*
National University of Singapore
Singapore
hruan@comp.nus.edu.sg

Yuntong Zhang*
National University of Singapore
Singapore
yuntong@comp.nus.edu.sg

Abhik Roychoudhury
National University of Singapore
Singapore
abhik@comp.nus.edu.sg

2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE) | 979-8-3315-0569-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSE55347.2025.00080

***Abstract*—Autonomous program improvement typically involves automatically producing bug fixes and feature additions. Such program improvement can be accomplished by a combination of large language model (LLM) and program analysis capabilities, in the form of an LLM agent. Since program repair or program improvement typically requires a specification of intended behavior - specification inference can be useful for producing high quality program patches. In this work, we examine efficient and low-cost workflows for iterative specification inference within an LLM agent. Given a GitHub issue to be resolved in a software project, our goal is to conduct iterative code search accompanied by specification inference - thereby inferring intent from both the project structure and behavior. The intent thus captured is examined by a reviewer agent with the goal of vetting the patches as well as providing a measure of confidence in the vetted patches. Our approach SpecRover is built on the open-source LLM agent AutoCodeRover. In an evaluation on the full SWE-Bench consisting of 2294 GitHub issues, it shows more than 50% improvement in efficacy over AutoCodeRover. Compared to the open-source agents available, our work shows modest cost ($0.65 per issue) in resolving an average GitHub issue in SWE-Bench lite. The production of explanation by SpecRover allows for a better "signal" to be given to the developer, on when the suggested patches can be accepted with confidence. SpecRover also seeks to demonstrate the continued importance of specification inference in automated program repair, even as program repair technologies enter the LLM era.**

I. Introduction

Automatic programming has long been an aspiration of software engineering research. It has inspired research in topics like program synthesis and repair. In recent times, automatic programming from natural language specifications has become somewhat more realistic due to the emergence of tools like GitHub Copilot. At the same time, the automatically generated code from Large Language Models (LLMs) suffers from errors and vulnerabilities [1], [2] and needs to be improved. For this reason, there has been a recent research focus on autonomous program improvement. The problem setting for autonomous program improvement involves solving of GitHub issues which would typically involve bug fixes or feature additions. Though these tools are employed on manually written software projects such as the recently proposed SWE-bench [3], they hold the promise of high quality trustworthy code construction from LLMs. Starting with the AI software engineer Devin [4] from a stealth startup, recently several autonomous program improvement tools such as AutoCodeRover [5] have been proposed for automatically solving GitHub issues (such as bug fixes or feature additions). By combining these technologies with code generation via GitHub Copilot, one can envision trustworthy code construction from LLMs.

Program improvement or program repair, typically requires capturing developer *intent* to guide the process. However, there is no formal specification of developer intent. The natural-language description of the developer intent is usually only available at a "higher level" - it captures the intended behavior of the entire software system. However to improve or repair specific components of a software system (where the error might have been localized) - one needs to infer specifications of the different components. A successful approach to program repair may thus involve specification inference - where by carefully analyzing the artifacts of the buggy program (such as program executions), we can infer snippets of the intended program behavior. The works on semantic program repair [6], [7] extract specifications via symbolic analysis of the given tests. Indeed, the existing literature on program repair [8] uses a given test-suite as developer intent, and hence is focused on avoiding test-data over-fitting. The works on semantic repair alleviate the over-fitting concern by inferring symbolic specifications from tests. Nevertheless, for the general problem of program improvement, the buggy program may or may not be accompanied by tests. Moreover, symbolic analysis based program repair has a high entry barrier for developers. For these reasons, recently autonomous program improvement using Large Language Models (LLMs) [5], [9], [10] has been studied.

In this work, we explore the role of program specifications thoroughly in LLM-guided autonomous software engineering workflows. To understand the intent of the developer and perform program improvement based on inferred specifications, we build our work on the publicly available AutoCodeRover [5] tool. The reason for this choice is strategic. In essence, AutoCodeRover takes the position that the structure of the program also captures a coarse encoding of the developer intent, and it tries to glean intent by analyzing (and searching over) the program structure; it performs code search on the project structure for fix localization. Thus, to build a workflow where we conduct high quality program improvement via iterative specification inference, we choose to build our work on AutoCodeRover. Our work looks into various sources of specifications such as function-level code summaries and testcases, apart from program structure. The core contribution thus lies in distilling the various specifications coming from

*Joint first authors, ordered alphabetically.

1558-1225/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSE55347.2025.00080

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

different sources into a single patch.

We thus present SpecRover, a progeny of AutoCodeRover, which conducts and exploits more powerful specification inference. Starting from a GitHub issue, it conducts code search guided by the program structure, as in AutoCodeRover. However, in the process of the code search, as it visits classes/methods, it also calculates and deposits the specifications of the classes/methods which would have allowed for remediation of the observable error, thereby capturing *intended* program behavior. The specifications gathered from the code search are deposited along with generated tests to a reviewer agent. The reviewer agent studies the specifications, generated tests, and natural language requirements to guide the patching. More importantly, the reviewer agent produces evidence of confidence in the reported patch.

Contributions: The core contributions of our work on SpecRover can be summarized as follows.

- *Specification Inference:* We examine the role of specification inference in LLM guided autonomous software engineering. Our work suggests iterative specification inference to guide patching in LLM oriented program repair workflows. Once the understanding of developer intent is accomplished via iterative specification inference, patch construction is a natural by-product of the inferred specification.
- *Suggesting patches with confidence:* We design a reviewer agent for code review which reconciles specifications, tests and natural language requirements. The reviewer agent can be seen as conducting a comprehensive patch validation. The reviewer agent can produce evidence of correctness of automatically generated fixes - such as explanation of patch, reproducer test, and the accumulated specifications from different code elements. These evidence can be maintained along with the automatically generated patches, to track future regressions.
- *Experimental evidence:* Our tool shows high efficacy, solving 19.3% issues in full SWE-bench and 31% on SWE-bench lite. We also balance other needs from LLM agents such as low cost ($0.65 per issue) and supporting higher precision/recall. Our tool is available open-source in Zenodo and will be publicly released upon acceptance.

regression in input validation of clustering metrics

```
from sklearn.metrics.cluster import mutual_info_score
import numpy as np

x = np.random.choice(['a', 'b'], size=20).astype(object)
mutual_info_score(x, x)
```

ValueError: could not convert string to float: 'b'
while

```
x = np.random.choice(['a', 'b'], size=20)
mutual_info_score(x, x)
```

works with a warning?
this worked in 0.21.1 without a warning (as I think it should)

(a) Issue statement.

```
def check_array(array, ...):
    dtype_orig = getattr(array, "dtype", None)
    ...
    if dtype_orig is not None and dtype_orig.kind == "O":
        # if input is object, convert to float.
        dtype = np.float64
    ...
    if dtype_numeric and array.dtype.kind == "O":
        array = array.astype(np.float64)
    ...
```

(b) Buggy function.

Intended behavior: The check_array method should validate the input array without attempting to convert arrays with dtype object containing non-numeric strings to float. ... If the array contains non-numeric strings, it should skip the conversion to float and proceed with the validation. ...

(c) Inferred intended behavior of the buggy function.

The patch attempts to handle non-numeric strings by catching a ValueError during the conversion to float. However, it does not correctly handle the case where the input contains non-numeric strings. The patch still tries to convert the array to float later in the code, which results in the same ValueError. The patch does not resolve the issue.

(d) Reviewer feedback on initial patch.

```
--- a/sklearn/utils/validation.py
+++ b/sklearn/utils/validation.py
@@ -459,8 +459,13 @@ def check_array(array, ...
     ...
     if dtype_orig is not None and dtype_orig.kind == "O":
-        # if input is object, convert to float.
-        dtype = np.float64
+        # if input is object, check if it contains non-numeric strings
+        try:
+            array.astype(np.float64)
+            dtype = np.float64
+        except ValueError:
+            # contains non-numeric strings, skip conversion
+            dtype = None
     ...

@@ -551,7 +556,11 @@ def check_array(array, ...
     ...
-            array = array.astype(np.float64)
+            try:
+                array = array.astype(np.float64)
+            except ValueError:
+                # contains non-numeric strings, skip conversion
+                pass
     ...
```

(e) Final patch generated by SpecRover.

Fig. 1: scikit-learn-15535 description and SpecRover artifacts.

II. Motivating Example

We now present the SpecRover approach via an example. The GitHub issue involved in this example is scikit-learn-15535[1], shown in Figure 1a. In the issue, two relevant code snippets are provided. According to the issue report, both snippets had worked without problem on an older version of scikit-learn, and it is expected that they continue to work on the current version. However, on the current version, the first snippet now crashes. The associated error information indicates that the crash occurred when scikit-learn mistakenly tries to convert a non-numeric array element into a float.

[1]https://github.com/scikit-learn/scikit-learn/issues/15534

To resolve the issue, SpecRover first identifies buggy program locations by exploring the program and retrieving relevant code. In this example, the identified buggy method is `check_array`, shown in Figure 1b. The method performs two conversions of string to float. The two conversions are the root cause of the failure reported in the issue, resulting in an exception when the involved string is non-numeric. SpecRover inferred a summary of *intended behavior* of the method `check_array` (shown in Figure 1c), which serves as a specification of how the method should be modified.

Next, the identified buggy method and its intended behavior are passed to our patching agent, which will write patches for

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

the method. At the first attempt, the patching agent wrote a partial patch, which only contains the first of the two hunks in Figure 1e, i.e., the patch only catches exceptions for the first conversion. Without further rectification of the patch, the issue could not be resolved. To vet the patch for potential mistakes, the initial patch is then passed to the reviewer agent. Apart from the patch, the reviewer agent also takes a reproducer test that reproduces the issue. The reviewer agent then runs the reproducer test on both the original program and the program repaired by the initial patch. By referring to the error information, the patch, and the issue statement, the reviewer agent is able to give the feedback as shown in Figure 1d. The feedback correctly indicates that the initial patch does not resolve the issue and can be rectified by catching exceptions for the other string conversion. Finally, the feedback is passed to the patching agent, which writes the correct patch shown in Figure 1e.

In this example, we illustrated how our reviewer agent provides feedback on an incorrect patch for our patch-writing agent. The feedback leads to a later rectification of the patch, and explains clearly why the initial patch is incorrect.

III. Methodology

A. Overview

Problem setup: Given a software codebase C and a natural language problem description D, the goal is to automatically derive a patch p (i.e. a set of code modifications) to C, such that the patched codebase C' satisfies the requirements in D. One example setup for D is GitHub issues, in which the issue description contains requirements for fixing a bug or adding a new feature.

In this paper, we drive autonomous program improvement with the help of program specifications. We try to acquire an understanding of the intended program behavior (the specification), which then allows us to produce high-quality patches that successfully resolve GitHub issues. Beyond producing high-quality patches, an additional benefit of understanding the specification is that it also serves as evidence as to *why* the patch is correct. The evidence holds promise in terms of easing software maintenance and engendering trust in the code. The key novelty of our approach lies in how we infer and utilize various forms of specifications. For an overview of all the specifications involved, we depict the general workflow of our approach SpecRover in Figure 2. In this figure, the inferred specifications are highlighted in yellow. We also highlight in blue all the LLM agents present in the workflow. As shown in Figure 2, the specifications are inferred in an iterative fashion: the agents take in specifications, possibly produced by other agents, and in turn infer new forms of specifications. This iterative process generates a variety of specifications, until a patch is generated and deemed correct by one of our agents that vets generated patches.

Specifically, as shown in Figure 2, the following specifications are inferred in sequence in SpecRover, which is given as input an issue statement and a software codebase.

1) The input issue statement is passed to a reproducer agent, which writes a *reproducer test* that reproduces the program fault reported in the issue.
2) The reproducer test, its execution results, along with the issue statement and the codebase, are passed to a context retrieval agent. The context retrieval agent explores the program codebase and identifies the relevant code to the issue. It eventually decides on a set of *buggy locations* that need patching.
3) The context retrieval agent also produces a *function summary* for every function encountered while exploring the program code. A function summary describes the intended behavior of a function in natural language, with respect to the current issue being solved.
4) The buggy locations, together with their corresponding function summaries, are passed to a patching agent, which tries to write a *patch* to resolve the issue.
5) The patch and the reproducer test are passed to a reviewer agent for scrutiny. The reviewer agent will produce a *reviewer feedback* if the patch is deemed incorrect; the patching agent will take in the reviewer feedback and try writing another patch. The reviewer feedback is a natural-language explanation of why the patch is incorrect and how it can be rectified. Likewise, a reviewer feedback for the reproducer test will be produced at the same time if the test is deemed incorrect.
6) If a patch is deemed correct by the reviewer agent, and there is an existing regression test suite available for the program, the patch will be checked via the regression test suite. If there is no regression, the patch will be accepted as the final patch. Otherwise, if some of the regression tests fail, we will retry the workflow up to a predefined number of times.
7) Finally, after multiple retries, there can be multiple patch candidates. A selection agent is invoked to select one final patch among the patch candidates, and give the reason why this patch is selected. The final patch, the reason for selection, and optionally the rest of the candidate patches will be sent to the user.

Among the specifications, the 3) function summary and 5) reviewer feedback are unique to SpecRover and unexplored by other LLM agents. These specifications have boosted the effectiveness of SpecRover in resolving software issues, because they fully exploit different kinds of software artifacts: the function summary exploits the program code behavior, and the reviewer feedback exploits both the code and the test.

B. Function Summary: Specification from Program

In this section, we first describe how the context retrieval agent gathers code context for the software issue to be resolved. We then discuss how SpecRover transforms the user intent in the issue description into program specifications for shorter code elements such as functions.

Existing LLM programming agents typically employ a *context retrieval* step to collect necessary code context related to the given issue from a large codebase. SpecRover follows

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

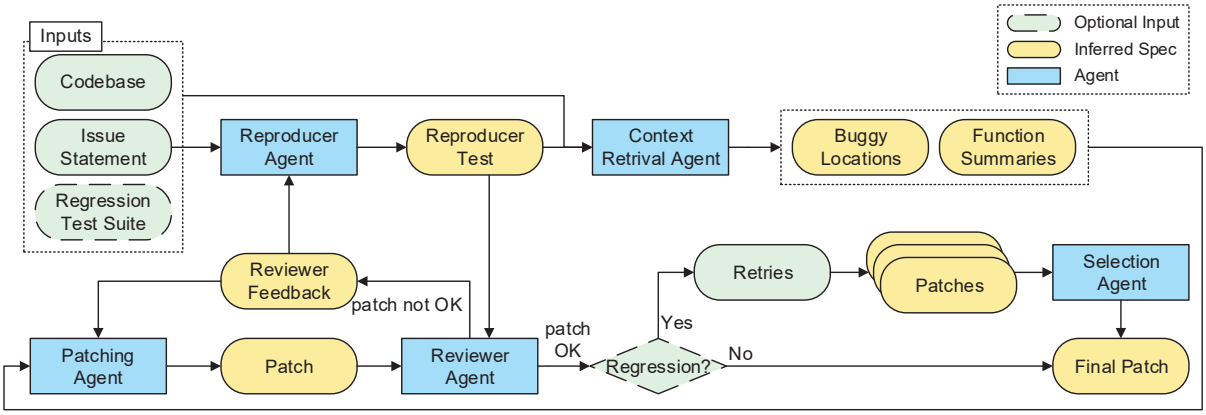

Fig. 2: Overall Workflow of SpecRover.

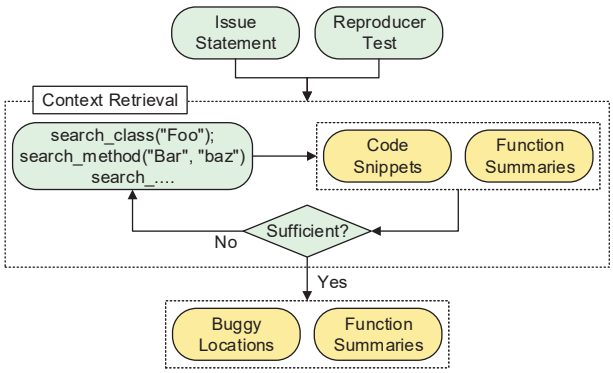

Fig. 3: Context retrieval in SpecRover.

the general architectural design of programming agents in its context retrieval stage, as shown in Figure 3. SpecRover conducts context retrieval by providing a set of APIs to the LLM for exploring the codebase. The LLM agent invokes the retrieval APIs to investigate the relevant code snippets in the program. The retrieved code forms the code context for the current to-be-resolved issue, which can contain definitions of the relevant classes and methods. After each round of retrieval API invocations, the LLM agent takes the code context collected so far and decides whether the context is sufficient for understanding and resolving the problem. If the context is deemed sufficient, the retrieval process will end, and the agent will decide on a set of *buggy locations*, which are sent to the patching agent for repairing. Otherwise, the retrieval process continues until a predefined threshold count is reached.

One key novelty in SpecRover is the *explicit* extraction of *function summaries* while collecting code snippets during context retrieval. In SpecRover, whenever a new code snippet is retrieved with an API and sent to the context retrieval agent, we explicitly prompt the agent to analyze the "intended behavior" of this code snippet in the current problem context. The intended behavior (or specification) is a concise natural-language summary of how a function should behave to meet the requirements specified in the high-level problem description. This function-level summary of intended behavior serves as a local specification to guide the patch construction. The system-level intended behavior specification given by the user (i.e. the issue description) is often on how the program should behave rather than how a unit function should behave. So we usually do not have the intended behavior of a function. Although the issue description may provide some "direction" on the intended behavior of a function - it is usually not sufficient to guide the patching agent. On the other hand, the extracted function-level specification (capturing the intended behavior of the function) serves as a more direct guide to the patching agent. Instead of giving a set of bug locations $\{L_1, L_2, ..., L_n\}$ to the patching agent to modify, SpecRover gives the pairs of bug locations and their corresponding local specification $\{(L_1, Spec_1), (L_2, Spec_2), ..., (L_n, Spec_n)\}$. The patching agent can then refer to the specifications of intended behavior and modify code at the function level (so as to achieve this intended behavior). Intuitively, our approach decomposes the repository-level issue solving task to several function-level code modification tasks, in which each function-level task has a natural language specification. LLMs have been extensively studied for function-level coding tasks and have shown promising results in function-level benchmarks such as HumanEval [11], [12] and MBPP [13]. Therefore, this task decomposition helps the patching agent of SpecRover which then has to solve smaller and more manageable tasks.

C. Reviewer Feedback: Reconciling Specifications

Another kind of specification inferred by SpecRover is the reviewer feedback. To be more precise, the reviewer feedback can be called a meta-specification: it is a reflection on the specifications inferred in previous steps. Concretely, given a patch and a reproducer test, the reviewer agent in SpecRover will provide feedback, which includes 1) a binary decision of whether the patch and the reproducer test are correct respectively; and 2) an explanation for the decisions.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

The reviewer feedback contributes to our specification inference practice in two ways. First, it makes the specification inference iterative. The reviewer feedback will be passed back to the patching agent and the reproducer agent, leading to improved patches and reproducer tests. Second, it reconciles the patch and the test. In this way, errors that are not obvious when examining the two separately can be revealed and rectified. What makes the reviewer feedback important is the absence of a suitable test suite. If a test suite was available for checking whether the issue has been resolved, patch correctness could be easily decided. In reality, however, issues occur when the program already passes the accompanying regression tests, which means that a high quality test-suite to check a generated patch (for the given issue) is usually not available.

To mitigate the lack of an issue-revealing test case, SpecRover writes a reproducer test via the reproducer agent. However, this test alone is not sufficient for deciding patch correctness. This is because the reproducer test can be *incorrect*, due to the non-determinism of the LLM, i.e., the test may fail a patch that actually conforms to the user intent. Besides, the reproducer test can also be *incomplete* description of intent, i.e., a patch may pass the test without completely resolving the issue. The limitation of the reproducer test derives from the fact that tests are a precise yet incomplete specification. To overcome the limitation, we make the observation that the natural-language issue statement is ambiguous in nature yet often contains richer information. Therefore, supplementing the test with an understanding of the issue statement is likely to help decide patch correctness. This is accomplished in the reviewer agent of SpecRover, which considers the issue statement as well as test to vet patch candidates.

Beyond deciding on patch correctness, the more important aspect of the reviewer feedback is an explanation of the decision made, which will help the patch agent rectify an incorrect patch. Further, for a correct patch, the explanation will help the user understand and accept the patch. The user can merge the reviewer feedback into the software together with the patch for future reference, which will help software maintenance in the long run.

```
An issue has been submitted. Engineer A has written a reproduction test for the issue.
Engineer B has written a patch for the issue. Your task is to decide whether the created
patch resolves the issue.
NOTE: both the test and the patch may be wrong.
Here is the issue: ...
Here is the test written by Engineer A: ...
Here is the result of executing the test on the original buggy program: ...
Here is the patch written by Engineer B: ...
Here is the result of executing the test on the patched program: ...
Think about
(1) whether the test correctly reproduces the issue, and
(2) whether the patch resolves the issue.
```

Fig. 4: Template prompt for the reviewer agent.

Our reviewer agent generates the feedback in two steps. First, the original program and the patched program are run on the reproducer test. In both runs, execution information including the output and the exit code are collected. The reviewer agent then provides the LLM with these execution information, along with the issue statement and the reproducer test. The LLM is prompted to decide whether the patch and the test are correct respectively, and to provide explanations for both decisions. Figure 4 shows the template of the prompt.

D. Patch Selection

As shown in Figure 2, a patch approved by the reviewer agent is checked through a regression test suite, which serves as an oracle for whether the patch breaks existing functionality of the program. However, in the setting of resolving GitHub issues, the regression test suite can be an inaccurate oracle, meaning that they can reject correct patches which resolve the issue. This is because the correct patch may inevitably modify existing functionalities of the program while resolving the issue, thus causing some of the existing regression tests to fail. For example, if the patch needs to modify the signature of an existing function `f` in order to resolve an issue, regression tests that invoke `f` will now fail. Since the correct patch can be rejected by the regression tests, we employ a patch *selection* process at the end of the workflow to select the most promising patches among the rejected candidate patches.

During the final patch selection phase, SpecRover goes beyond the test cases and employs a *selection agent* to choose a patch based on the natural language issue description. All candidate patches that failed some tests are presented to the selection agent, together with the issue description. The selection agent is instructed to analyze the root cause of the issue, think about how the issue can be possibly resolved, and select a patch that best addresses the issue. This natural language-guided patch selection can recover correct patches that are mistakenly filtered out by an inaccurate test suite. It exploits the natural language issue report as that captures the most up-to-date intents from users/developers. While making a choice among the candidate patches, the selection agent also explicitly states a reason why it chooses a particular patch among the candidate patches. This "reason for selection" can be given as *evidence* together with the final patch.

E. Evidence

SpecRover is designed to not only generate a patch to resolve the issues in software repositories, but also to provide the inferred specifications as evidence for why a patch was selected. Specifically, along with the final patch, the following artifacts can be the outputs of SpecRover as well:

- Buggy locations and their intended behaviors.
- The reproducer test written by the reproducer agent.
- The reason why the final patch was approved by the reviewer agent (if the patch is approved by the reviewer and the regression test suite).
- The reason why the final patch is selected by the selection agent (if there are multiple candidate patches which cannot be differentiated by the tests).

The benefits of generating evidence are threefold. First, these artifacts can guide the LLM agents in constructing higher quality patches, as discussed in Section III-B and III-C. Second, the natural language artifacts can assist the developers in understanding the auto-generated patches more quickly. Before examining the actual patch, developers can gain a

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

preliminary understanding of the changes by reviewing the reasons for approval or selection and the intended behaviors for the buggy locations. Last but not least, the evidence can be integrated into the software repository and can evolve with it. The developers can integrate the reproducer test for this issue as part of the test-suite of the program. Reasons for patch approval/selection can become parts of the commit message when the auto-generated patch is committed to the repository. Overall, we propose SpecRover as a programming agent that not only automatically generates code improvements but also produces evidence that enriches the software system lifecycle.

IV. Experimental Setup

We address the following research questions:

RQ1: What is the efficacy of SpecRover in resolving issues?
RQ2: What level of confidence can developers get from the patch and specifications produced by SpecRover?
RQ3: What is the quality of the specification produced by SpecRover as evidence?

a) Benchmark: We evaluate the efficacy of SpecRover on SWE-bench [3], a widely-used benchmark for autonomous program improvement consisting of 2294 real-world GitHub issues. For each issue, the only input for SpecRover is the issue statement and the buggy codebase. Note that the regression test suite used by SpecRover is part of the *buggy* program; we do not access any code or test that is added by the developer in the fixed version of program.

b) Baselines and Evaluation Metrics: For RQ1, we compare with the state-of-the-art systems that target the repository-level issue solving task. In our comparison, we include all the open-source software engineering agents which have reported results on SWE-bench. The baseline tools include:

- AutoCodeRover [5]. AutoCodeRover employs a set of program structure-aware APIs to gather relevant code context. It optionally integrates debugging techniques such as Spectrum-based Fault Localization to sharpen the context.
- SWE-agent [9]. SWE-agent designs an agent-computer interface, which defines the possible actions taken by an agent to edit code, navigate the codebase, and execute tests.
- AppMap Navie [14]. Navie uses a retrieval-augmented generation (RAG) based approach to construct the code context, and performs an explicit planning step before generating code changes [15].
- OpenDevin [16]. OpenDevin's CodeActAgent tackles the tasks by having a general action space, where the agent is allowed to execute arbitrary Python and bash commands inside a sandbox environment.
- Aider [17]. Aider constructs a repository map which helps the LLM to understand the repository context. It also uses the regression test suite as a harness to retry the task.
- Moatless Tools [18]. Moatless Tools builds an agentic loop that functions as a finite state machine and transitions between states. It focuses on building a set of good tools for the agent instead of relying on the agent for reasoning.
- Agentless [10]. Agentless is a concurrently pursued (currently unpublished) arXiv report which employs a fixed two-phase approach of localization and repair, without allowing the LLM to decide on actions or utilize tools.

We report pass@1 efficacy on SWE-bench for all tools. For each issue, SWE-bench has a set of acceptance tests written by the developers to evaluate the patch correctness. These acceptance tests are not used by the tools when generating patches. We follow the official SWE-bench evaluation criteria - if the single patch generated by a tool passes the SWE-bench acceptance tests for the issue, the issue is considered as resolved.

c) Implementation and Parameters: We implemented SpecRover on top of the AutoCodeRover codebase and reuse its context retrieval APIs. We implemented new features unique to SpecRover such as function summary extraction as part of the context retrieval process. Other unique features such as patch reviewing and selection are implemented as new LLM agents. SpecRover supports multiple LLMs as backend. In our experiments, we used the Claude 3.5 Sonnet as the main foundation model, and only switch to OpenAI GPT-4o for a task if that task encounters an API error when invoking the Claude remote APIs. We set maximum retries on regression test suite failures to be 3.

d) Randomness of LLMs: LLMs are inherently random in its output generation, which may threaten the validity of LLM-based coding agents including SpecRover. We address this by setting the model temperature to 0, so that the model output is more deterministic.

e) Manual Inspection of Results: For a better understanding of the experimental results, we perform manual inspection to obtain certain data, e.g., patch overfitting rate. All manual inspections were independently conducted by two authors and subsequently cross-checked. Discrepancies were resolved through consultation with a third colleague from our group.

V. Evaluation

A. RQ1: Overall Efficacy of Task Resolving

We first evaluate the overall efficacy of SpecRover in resolving repository-level tasks. We report the efficacy of SpecRover on both SWE-bench (consisting of 2294 real-world GitHub issues), and SWE-bench Lite (which is a subset of SWE-bench consisting of 300 issues). For the baseline tools, we compare with their corresponding reported efficacy. If a tool supports different configurations (e.g. different LLMs as the backend), we compare with the configuration with the highest efficacy.

Result: Table I shows the efficacy of issue resolving in both SWE-bench and SWE-bench Lite. Overall, SpecRover achieves the highest efficacy among all the open-source tools in both SWE-bench and SWE-bench Lite. In SWE-bench Lite, compared to the previously top-performing group of tools which resolved approximately 26% to 27% of the issues, SpecRover improved the efficacy to 31%. This efficacy improvement is also evident in the full SWE-bench, where SpecRover improved the efficacy from 14.60% to 19.31%.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restriction

TABLE I: Comparison of efficacy/efficiency/cost on SWE-bench and SWE-bench Lite.

Tool	LLM	Resolved%	Avg. Time (s)	Avg. Cost ($)
Efficacy on SWE-bench (size=2294)				
AutoCodeRover	GPT-4	12.42% (285)	248	0.45
SWE-Agent	GPT-4	12.47% (286)	-	1.59
AppMap Navie	GPT4o	14.60% (335)	-	-
SpecRover	Sonnet-3.5+GPT-4o	**19.31%** (443)	362	0.72
Efficacy on SWE-bench Lite (size=300)				
SWE-Agent	GPT-4	18.00% (54)	-	1.67
AutoCodeRover	GPT-4	19.00% (57)	195	0.43
AppMap Navie	GPT-4o	21.67% (65)	-	-
OpenDevin	Sonnet-3.5	26.00% (78)	-	1.10
Aider	GPT-4o+Opus-3	26.33% (79)	-	-
Moatless Tools	Sonnet-3.5	26.67% (80)	71	0.17
Agentless	GPT-4o	27.33% (82)	-	0.34
SpecRover	Sonnet-3.5+GPT-4o	**31.00%** (93)	309	0.65

'-' indicates data is not publicly available.

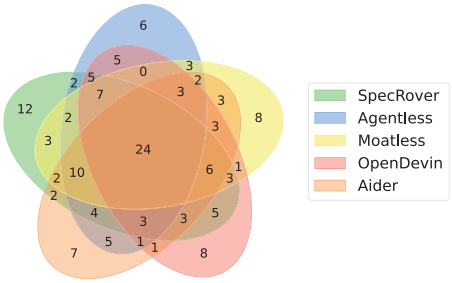

Fig. 5: Number of uniquely resolved issues by the top performing open-source tools on SWE-bench Lite.

Figure 5 illustrates the number of uniquely resolved issues by SpecRover and other tools in SWE-bench Lite. For clarity, this figure includes only the top five performing tools from Table I. SpecRover uniquely resolved 12 issues, the highest number of uniquely resolved issues among all the tools. Among the 12 uniquely resolved issues, SpecRover resolved six of them by generating only one patch, demonstrating that the inferred function summary can effectively guide the LLM to generate correct patches. For the other six issues, SpecRover deemed the first generated patch as incorrect from the reviewer agent and the regression test suite. In this case, the patches are iteratively refined based on the reviewer feedback and the test results, and eventually the correct patch is selected at the end of the workflow.

Data Memorization: An issue that can bias the evaluation of LLM agent-generated patches is *data memorization* [19]. Data memorization occurs when an LLM deals with a program that exists in its training set. To evaluate the risk of data leakage, we specifically count the patches generated by SpecRover that are syntactically identical to the ground-truth patches. Our counting shows that in SWE-Bench Lite, SpecRover generated the ground-truth patch only for 9 issues, accounting for 10% of the 93 resolved issues. The result shows that data memorization occurs very infrequently for SpecRover.

Time and Cost: We also report the average time taken and average costs for each issue in Table I. For each tool, we include the time and cost statistics in Table I if these information was publicly reported or can be calculated from their publicly available execution traces. On average, SpecRover costs $0.65 USD to generate patches for each issue in SWE-bench Lite, achieving the highest efficacy with a relatively low cost. We further investigate the 93/300 issues resolved by SpecRover in SWE-bench lite. For the resolved issues, SpecRover only costs $0.36 USD per issue to generate the correct patch, suggesting that the resolved issues requires less retries and less API calls to the LLM in general. Time-wise, SpecRover spends an average of 309 seconds (i.e. 5.15 minutes) on each issue, which includes the time for executing the reproducer and the regressions tests in the project. According to a recent study, most developers accept automated program repair tools which takes less than 30 minutes [20]. SpecRover requires approximately 5 minutes, which we deem acceptable.

Patch Correctness: A patch that passes a given test suite is not necessarily correct, because the test suite is an incomplete specification. This problem is known as *patch overfitting* [21] in program repair. For a more accurate evaluation of the generated patches, we manually compared the SpecRover-generated patch with the developer patch for each resolved issue. Our manual investigation confirmed that 54.8% (51/93) of the patches that pass the test suites are semantically equivalent to the ground-truth patches. We also observed that 34 out of the 42 (=93-51) overfitting patches (i.e., the patches that pass the test suite but are not semantically equivalent to ground truth) modify the same methods as the ground-truth patch, and only the specific modification is semantically different from the ground truth. This implies that even overfitting patches produced by SpecRover are useful in indicating a fix location. Adding up the semantically equivalent patches and the overfitting patches that have the correct fix location, a total of over 90% of the test-passing patches prove useful for issue

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

resolution.

Reasons for Failure: We examined the 207 tasks that are not resolved by SpecRover in SWE-Bench Lite.These failure cases break down into three cases:

1) Ambiguous issue description (as labeled by SWE-bench Verified [22]): 107 (51.7%)
2) Incorrect fix location: 61 (29.5%)
3) Incorrect code modification: 39 (18.8%)

Case 1 is "hard or impossible to solve" [22] without further information. Case 2 is more frequent than Case 3, implying that SpecRover can potentially have significant improvement by leveraging more fix localization techniques. For Case 3, in most such cases (26/39), we observed that the gist of the patch generated by SpecRover is actually correct, but some code detail is wrong. To reduce this kind of failures, more test generation techniques can be leveraged within our agent.

B. RQ2: Utility of autonomous SE, confidence in results

Although the efficacy in resolving issues is an important aspect of autonomous program improvement, it is not the sole purpose of such a technique. Rather, the efficacy is a means to an end – to reduce human effort in software maintenance. To this end, a program improvement technique must not only have high efficacy, but also minimize the effort required of an end user to use the technique. The effort is related to two metrics: 1) *signal-to-noise ratio*, i.e., the ratio of correct to incorrect patches presented to a user; and 2) the difficulty of examining each auto-generated patch that is suggested.

We have designed SpecRover to reduce both of these efforts. First, to reduce the number of incorrect patches that a user may examine, we use the reviewer agent to decide the correctness of the generated patch and the reproducer test. The user can choose to examine the generated patch only when both the patch and the reproducer test are deemed correct by the reviewer agent. The accuracy of the reviewer decisions are measured in RQ2. Second, to make it easy for a user to examine each patch, SpecRover provides a variety of evidence to help understand the patch, as discussed in Section III-E. The quality of the evidence will be discussed in RQ3.

There can be four different scenarios when the reviewer decision is viewed in relation to the actual correctness of the patch. For convenience, we say a patch is *accepted* when the reviewer agent decides that both the generated patch and the reproducer test are correct. With this, we discuss the following four scenarios:

- TP (true positive): Patch is accepted and correct;
- TN (true negative): Patch is not accepted and incorrect;
- FP (false positive): Patch is accepted but incorrect;
- FN (false negative): Patch is not accepted but correct.

Table II lists the frequency that each scenario occurred in our experiment. The table counts in 119 tasks in SWE-bench Lite for which a reproducer test was generated. In the table, we also calculate the accuracy, precision, and recall of the reviewer decisions. Out of the 119 tasks, there are 26 TP's and 51 TN's, i.e., as many as 64.7% (accuracy) of the reviewer decisions were consistent with the actual correctness of the patch. The recall was also over 60%, meaning that the majority of the generated correct patches were recognized by the reviewer agent.

TABLE II: Reviewer decisions on SWE-Bench lite.

Category	# Tasks
TP	26
TN	51
FP	26
FN	16
Total	119
Accuracy = (TP+TN) / Total	64.7%
Precision = TP / (TP+FP)	50.0%
Recall = TP / (TP+FN)	61.9%

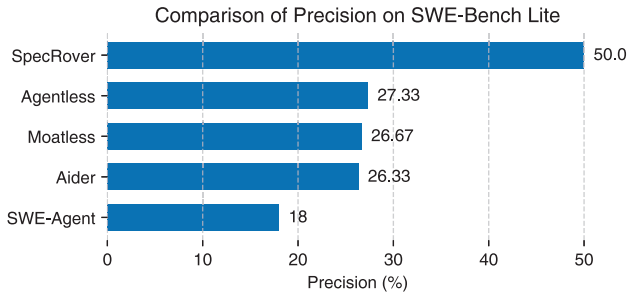

Fig. 6: Comparison of patch precision on SWE-bench Lite. If SpecRover does not use regression tests for patch validation, precision reduces only slightly to 48.3%.

A metric of particular interest to program improvement techniques is the precision. The precision is defined as TP/(TP+FP), i.e., the proportion of correct patches in all the patches offered by a technique. It is directly related to user effort required to examine generated patches. For SpecRover, the precision is 50.0%, as calculated in Table II. To put the precision in perspective, we compare the precision of SpecRover with that of other baseline tools in Figure 6. The precision of the baseline tools is the same as their pass@1 efficacy reported in Table I, since these tools indiscriminately present every generated patch to a user. As can be seen in Figure 6, the precision of SpecRover is higher than 1.8x that of Agentless, which has the second highest precision. The high precision of SpecRover indicates a much lower cognitive load imposed on the user, compared to other techniques. Moving forward, we suggest paying attention to agent precision.

C. RQ3: Quality of Specifications produced

In this section, we investigate the quality of evidence generated by SpecRover. The high-quality evidence allows a developer to easily integrate auto-generated patch into an existing codebase.

1) Function Summaries: We manually investigated the natural-language specifications generated by SpecRover for SWE-bench Lite issues. To obtain some approximation of "ground truth" for the generated specifications, we extracted

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restriction

skipping: --runxfail breaks pytest.mark.skip location reporting
pytest versions: 5.4.x, current master
When `@pytest.mark.skip` / `skipif` marks are used to skip a test, for example

```
import pytest
@pytest.mark.skip
def test_skip_location() -> None:
    assert 0
```

the expected skip location reported should point to the item itself, and this is indeed what happens when running with `pytest -rs`:

```
SKIPPED [1] test_it.py:3: unconditional skip
```

However, adding `pytest -rs --runxfail` breaks this:

```
SKIPPED [1] src/_pytest/skipping.py:238: unconditional skip
```

The `--runxfail` is only about xfail and should not affect this at all.

Hint: the bug is in `src/_pytest/skipping.py`, the `pytest_runtest_makereport` hook.

(a) Issue statement.

```
def pytest_runtest_makereport(item: Item, ...):
    ...
    elif item.config.option.runxfail:
        pass  # don't interfere
    elif call.excinfo and isinstance(call.excinfo.value, ...):
        ...  # (3 lines)
    elif not rep.skipped and xfailed:
        ...  # (14 lines)
    elif ...:
        # calculate the line number
```

(b) Buggy code snippet.

```
--- a/src/_pytest/skipping.py
+++ b/src/_pytest/skipping.py
@@ -270,33 +270,34 @@ def pytest_runtest_makereport(item: Item, ...):
def pytest_runtest_makereport(item: Item, ...):
    ...
-    elif item.config.option.runxfail:
-        ... # (22 lines)
+    elif not item.config.option.runxfail:
+        if call.excinfo and isinstance(call.excinfo.value, ...):
+        ... # (20 lines)
+    if ...: # was elif
         # calculate the line number
+        # ... # (4 lines)
```

(c) Correct patch generated by SpecRover.

The patch correctly addresses the issue by modifying the logic in the pytest_runtest_makereport function. It ensures that the skip location is reported correctly regardless of whether the --runxfail option is used. The patch moves the handling of skip location correction outside the conditional block for --runxfail, which fixes the reported issue.

(d) Reviewer feedback.

Fig. 7: pytest-7432 description and SpecRover artifacts.

Issue: Wrong Derivative kind attribute
I'm playing around with the `kind` attribute.
The following is correct:

```
A = MatrixSymbol ('A', 2, 2)
i = Integral (A, x)
i.kind      # MatrixKind(NumberKind)
```

This one is wrong:

```
d = Derivative (A, x)
d.kind      # UndefinedKind
```

(a) Issue statement.

Buggy Locations
Class Derivative in
file sympy/core/function.py

Intended behavior:
Add a `kind` property to the `Derivative` class that returns the appropriate kind for derivatives. This may involve checking the kind of the expression being differentiated and the differentiation variables.

(b) Location and summary.

```
try:
    A = MatrixSymbol('A', 2, 3)
    i = Integral(A, x)
    assert i.kind == 'MatrixKind(NumberKind)',
    f"Expected 'MatrixKind(NumberKind)', got {i.kind}"
    d = Derivative(A, x)
    assert d.kind == 'MatrixKind(NumberKind)',
    f"Expected 'MatrixKind(NumberKind)', got {d.kind}"
except AssertionError as e:
    # issue reproduced
    print_stacktrace(e)
    raise
```

(c) Generated reproducer test.

```
--- a/sympy/core/function.py
+++ b/sympy/core/function.py
 ...
+    @property
+    def kind(self):
+        return self.expr.kind
+
     def __new__(cls, expr, *variables, **kwargs):
 ...
```

(d) Correct patch generated by SpecRover.

Patch is correct.
Reason: The patch correctly adds a `kind` property to the `Derivative` class that returns the `kind` of the expression being differentiated.

Test is wrong.
Reason: The test incorrectly compares the `kind` attribute to a string `MatrixKind(NumberKind)`. The `kind` attribute is an object, not a string, and should be compared directly to the expected `kind` object.

(e) Reviewer feedback.

Fig. 8: SymPy-21614 description and SpecRover artifacts.

the titles of Pull Requests that developers wrote when they fixed each issue. The PR title usually summarizes the fixes made in the PR in one sentence. In this investigation, we include only the issues for which SpecRover generated a patch at the same methods as the developer's patch. Moreover, we exclude the issues whose developer-written PR title is too brief (e.g. "Fixes issue #..."). For the included issues, we manually compare the specification generated by SpecRover for the patched methods, with the PR title written by the developer. For 72/101 (71%) inspected issues, the specification generated by SpecRover covers similar intent to the human-written PR title, which shows that the generated specification serves as a good summary of the intended behavior. The generated specifications are generally an extended version of the PR title and additionally describe how the code should be modified.

2) Reviewer Feedback: We cannot conduct a quantitative examination on the quality of the reviewer feedbacks on the generated patches, since there is no "ground truth". Therefore, we illustrate the quality of the feedbacks with two examples.

a) Reviewer Feedback as Summary: In the first example, we show that the reviewer feedback can serve as a concise summary of a generated patch. The summary describes the behavior of the patch at a high level. Therefore, a developer can understand the generated patch faster by reading the summary before examining the details of the patch. Besides, after the developer accepts the patch and decides to merge it into existing code, the summary constitutes a good commit message, so that the developer does not need to write one. From a developer's perspective, the whole process is very much like reviewing a pull request, which is already part of a developer's day-to-day workflow. The specific issue involved in this example is pytest-7432[2]. The issue statement is shown in Figure 7a, which reports that pytest (a python testing framework) would miscalculate a line number in its output when an irrelevant option (`runxfail`) is enabled. The bug is caused by the code shown in Figure 7b. As can be seen, the calculation of the line number is wrongly placed in a branch that is mutaully exclusive with the `runxfail` branch. Therefore, the calculation is wrongly skipped when the option is enabled.

[2]https://github.com/pytest-dev/pytest/issues/7392

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

To resolve the issue, SpecRover was able to locate the relevant code and produce the correct patch. An abridged version of the patch is shown in Figure 7c. It correctly addresses the issue by moving the line number calculation to a branch unaffected by the `runxfail` option. However, the patch might not be immediately understandable to a developer, because it changes as many as 51 lines in the original program (though most of the changes just involve the indentation level). Fortunately, the understanding of the patch can be eased by the reviewer feedback. The reviewer agent was able to identify the patch as correct and produced the feedback in Figure 7d. It properly summarized that the patch just moved the calculation to another branch. Using this summary, the developer would easily understand the patch and accept it.

b) Tolerance of Incorrect Tests: Another advantage of the reviewer is enhanced tolerance of incorrect automatically generated tests. We illustrate this advantage with the example of SymPy-21614[3], where SpecRover rejects an incorrectly written test while approving a correct patch. The issue statement and the buggy location identified by SpecRover are shown in Figure 8a and 8b. The issue mentioned an unexpected behavior of the `kind` attribute. After its context retrieval stage, SpecRover correctly identifies that the buggy location is in the `Derivative` class, and that its intended behavior is to have an additional `kind` property. Figure 8c and 8d show the automatically generated reproducer test and patch for this issue. In this case, the reproducer test is incorrect - the assertions compare an object with a string, which always evaluate to `False`. If this reproducer test is used solely to determine the correctness of the generated patches, any patch, even a correct one, will be rejected. However, since the reviewer agent in SpecRover simultaneously examines both the reproducer test and the patch without assuming either is correct, it is capable of rejecting the reproducer test while approving the patch. Figure 8e shows the reviewer agent's decision and comments towards the test and patch. The reviewer identifies that the assertions in the reproducer are written incorrectly, thereby rejecting the reproducer. On the other hand, the reviewer correctly approves the patch despite the presence of an unreliable test. The correct patch, along with the reasons for rejecting the reproducer, can be sent to the developer. The developer can then integrate the patch into the codebase. Additionally, the developer can revise the "almost correct" reproducer test based on the feedback provided by the reviewer agent.

VI. Case Study: Security Vulnerability Repair

Although SpecRover is initially designed to resolve GitHub issues in Python repositories, it can be easily adapted for program improvements in other application domains and in other programming languages. We demonstrate how SpecRover fixes security vulnerabilities in C programs, through an example challenge problem from the DARPA AI Cyber Challenge

```c
// file: net/tipc/crypto.c
static bool tipc_crypto_key_rcv(struct tipc_crypto *rx, ...) {
...
    /* Copy key from msg data */
    skey->keylen = keylen;
    memcpy(skey->alg_name, data, TIPC_AEAD_ALG_NAME);
    memcpy(skey->key, data + TIPC_AEAD_ALG_NAME + sizeof(__be32),
                skey->keylen);
...
```

(a) Buggy code snippet.

```
BUG: KASAN: slab-out-of-bounds in tipc_crypto_msg_rcv+0x398/0x6e0
Write of size 1220 at addr ff11000004579024
Call Trace:
    ...
[    8.941611]  kasan_report+0xad/0x130
[    8.941796]  ? tipc_crypto_msg_rcv+0x398/0x6e0
[    8.942020]  kasan_check_range+0x35/0x1c0
[    8.942223]  memcpy+0x39/0x60
[    8.942379]  tipc_crypto_msg_rcv+0x398/0x6e0
    ...
```

(b) Vulnerability report generated after executing the PoC.

File: `net/tipc/crypto.c`, Method: `tipc_crypto_key_rcv`

Intended behavior: The `tipc_crypto_key_rcv` function should:
1. **Validate the Message Data Size and Key Length**:
Before performing any memory operations, the function should ensure that the message data size and key length are within valid bounds to prevent out-of-bounds memory access.
2. **Safe Memory Operations**:
The `memcpy` operations should be performed only if the source and destination buffers are within valid memory bounds.
...

(c) Buggy location and function summary by SpecRover.

```diff
--- a/net/tipc/crypto.c
+++ b/net/tipc/crypto.c
@@ -2285,12 +2285,24 @@ static bool tipc_crypto_key_rcv ...
    u32 size = msg_data_sz(hdr);
    u8 *data = msg_data(hdr);
    unsigned int keylen;
+
+   /* Validate message data size */
+   if (size < TIPC AEAD ALG NAME + sizeof(  be32)) {
+       pr err("%s: invalid message data size\n", rx->name);
+       return false;
+   }
    ...
+   /* Validate key length */
+   if (keylen > size - (TIPC AEAD ALG NAME + sizeof(__be32))) {
+       pr err("%s: invalid key length\n", rx->name);
+       return false;
+   }
+
    spin_lock(&rx->lock);
    if (unlikely(rx->skey || (key_gen == rx->key_gen && ...))) {
        pr_err("%s: key existed <%p>, gen %d vs %d\n", rx->name,
```

(d) Correct patch generated by SpecRover.

Patch is correct.
Reason: The patch addresses the root cause of the slab-out-of-bounds issue by adding validation checks for the message data size and the key length before accessing the data. This ensures that the memory access is within the allocated bounds, preventing the out-of-bounds write that was causing the crash. The validation checks are appropriately placed and cover the necessary conditions to prevent the vulnerability without introducing new issues or making significant functional changes.

(e) Reviewer feedback.

Fig. 9: Linux Kernel Exemplar Challenge in DARPA AIxCC.

(AIxCC) in 2024 [23]. The AIxCC is a two-year competition organized by DARPA and ARPA-H to encourage the development of novel cyber-reasoning systems to safeguard critical software. The AIxCC has publicly released exemplar challenges, where each challenge consists of a software project with a vulnerability. The task is to have an autonomous system to find and fix the vulnerability. Each exemplar challenge also contains a Proof-of-Concept (PoC) input file that triggers the vulnerability, so we use this PoC to show how SpecRover can be used to fix the vulnerability. Figure 9 shows one exemplar challenge, which is a buffer overflow[4]. It occurs in the Linux networking module for the Transparent Inter-Process

[3]https://github.com/sympy/sympy/issues/21604

[4]CVE-2021-43267 re-introduced to Linux kernel 6.1.54

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio

Communication (TIPC) protocol, and allows remote attackers to cause denial-of-service or disclosure of sensitive information. Specifically, when the user-supplied sizes in the message body are invalid for the received messages, a buffer overflow happens with the `memcpy` call, as shown in Figure 9a. This vulnerability has been triggered by a PoC, which results in a *vulnerability report* as shown in Figure 9b.

SpecRover fixes this vulnerability by first analyzing the vulnerability report, similar to how it resolves GitHub issues. It conducts context retrieval, and decides on the buggy locations and intended behaviors as shown in Figure 9c. Even though the vulnerability report only contains the call trace and minimal description of the bug (e.g., "slab-out-of-bounds"), SpecRover can infer the intended local behavior at the function level. Based on the intended behavior, SpecRover generated the patch in Figure 9d, which correctly fixes the vulnerability inserting additional checks before the dangerous memory operation. The reviewer agent approved the patch with the comments shown in Figure 9e, with which the developers can gain an initial understanding of the patch before closely examining the changed code.

VII. Related Work

Automated program repair (APR) [8], [24] is a well studied research area in software engineering. Given a buggy program P, and a test-suite T, automated program repair attempts to (minimally) modify P to a program P' which passes the given test-suite T. APR techniques involve metaheuristic search [25], semantic analysis [6], machine learning [26], or a combination of different techniques. APR can also be used to rectify automatically generated code from LLMs, see e.g. [1].

The recent interest in prompt engineering as well as agent based solutions has somewhat evolved the research in program repair. LLM agents try to combine the power of LLM with program analysis and test execution reasoning. Thus LLM agents can combine LLMs with test generation, static and dynamic analysis as well as specification inference. In the recent past, lot of LLM based approaches have been proposed for solving software "issues" described in natural language, including [4], [5], [9], [10]. Among these our work is thematically closest to the work of AutoCodeRover [5]. Like AutoCodeRover. we take the position that program modifications like bug fixing are best aided by inference of the developer intent. AutoCodeRover infers the developer intent only from the software project structure. In contrast, SpecRover is more general and is capable of inferring specifications from different sources including program structure, program behavior, tests and so on. Furthermore, SpecRover focuses on giving an explanation of the produced patches.

Previous works have studied function-level specification inference by means of LLMs [27]–[29]. However, these works either focus on generating specifications for simple one-function or one-class programs [27], [28], or assume that a target function is provided [29]. In contrast, SpecRover targets large programs and does not assume the target functions to generate specifications for are given.

Since both tests and patches generated in an autonomous workflow can be unreliable, we additionally consider the natural-language issue description when judging their correctness. The judgement process is concretized as the reviewer agent. Compared to the previous works that discover additional specification based on user intent [30], we do not rely on interactive user feedback. Instead, we utilize the high-level natural-language description as additional feedback. Furthermore, compared to previous works that evaluate the generated code with an LLM-based reviewer [31], our reviewer agent is designed for the setup where both natural-language instructions and unreliable tests are present.

VIII. Perspectives

Owing to the growth of LLM-based automatic programming (see [32] for a recent summary), there exists interest in autonomous program improvement technologies. We propose SpecRover with the perspective of autonomously producing patches which are suggested with confidence (thus developers can confidently accept them) and come with explanations. The technical innovations supporting SpecRover are the specification inference to guide patching, and the rigorous vetting of patches via our reviewer agent. Our work on SpecRover seeks to put the matter of quality of patches produced by LLM agents into the research community's attention, whereas other works are mostly focusing on the agent efficacy.

Moving forward, we envision that LLM agents will need to improve the precision and recall. Specifically, LLM agents will need to vet the produced patches. The vetting needs to be accompanied by sophisticated test generation, so that there is a rich test-suite to check the produced patches. Thus, we need to take the viewpoint of a developer using an LLM agent, who would be concerned about (a) efficacy, (b) cost, and, most importantly (c) *signal-to-noise ratio* in the agent output.

Data Availability

We share full public access of the source code and experimental artifacts of SpecRover at https://zenodo.org/doi/10.5281/zenodo.13161650.

We are continuously improving the efficacy and usability of SpecRover (i.e., AutoCodeRover-v2). At the time of paper acceptance (Nov 2024), AutoCodeRover-v2 has achieved an efficacy of 37.3% on SWE-bench Lite and 46.2% on SWE-bench Verified. We have also packaged AutoCodeRover-v2 as a GitHub bot offering one-click issue resolution [33]. The latest source code, experimental results, and news updates of AutoCodeRover-v2 can be found at our GitHub repository[5] and website[6].

Acknowledgments

This work was partially supported by a Singapore Ministry of Education (MoE) Tier 3 grant "Automated Program Repair", MOE-MOET32021-0001.

[5]https://github.com/nus-apr/auto-code-rover
[6]https://www.autocoderover.net/

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restriction

REFERENCES

[1] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *International Conference on Software Engineering (ICSE)*, 2023.

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *IEEE Symposium on Security and Privacy (SP)*, 2022.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[4] C. Labs, "Devin, ai software engineer," April 2024. [Online]. Available: https://www.cognition-labs.com/introducing-devin

[5] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *ACM International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[6] H. D. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, "Semfix: Program repair via semantic analysis," in *International Conference on Software Engineering (ICSE)*, 2013.

[7] S. Mechtaev, J. Yi, and A. Roychoudhury, "Angelix: Scalable multiline program patch synthesis via symbolic analysis," in *International Conference on Software Engineering (ICSE)*, 2016.

[8] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, 2019.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," 2024.

[10] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying llm-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[12] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[13] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[14] AppMap, "Appmap navie," July 2024. [Online]. Available: https://appmap.io/product/appmap-navie

[15] K. Gilpin, "How appmap navie solved the swe bench ai coding challenge," June 2024. [Online]. Available: https://dev.to/appmap/how-appmap-navie-solved-the-swe-bench-ai-coding-challenge-20an

[16] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, "Opendevin: An open platform for ai software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[17] P. Gauthier, "Aider," July 2024. [Online]. Available: https://github.com/paul-gauthier/aider

[18] A. Örwall, "Moatless tool," July 2024. [Online]. Available: https://github.com/aorwall/moatless-tools

[19] I. Magar and R. Schwartz, "Data contamination: From memorization to exploitation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Association for Computational Linguistics, 2022, pp. 157–165. [Online]. Available: https://doi.org/10.18653/v1/2022.acl-short.18

[20] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust enhancement issues in program repair," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2228–2240.

[21] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015, Bergamo, Italy, August 30 - September 4, 2015*, E. D. Nitto, M. Harman, and P. Heymans, Eds. ACM, 2015, pp. 532–543. [Online]. Available: https://doi.org/10.1145/2786805.2786825

[22] N. Chowdhury, J. Aung, C. J. Shern, O. Jaffe, D. Sherburn, G. Starace, E. Mays, R. Dias, M. Aljubeh, M. Glaese, C. E. Jimenez, J. Yang, K. Liu, and A. Madry, "Introducing swe-bench verified," Aug 2024, accessed: 2024-11-27. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[23] DARPA, "Ai cyber challenge (aixcc)," July 2024. [Online]. Available: https://aicyberchallenge.com/

[24] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys*, vol. 51, no. 1, 2018.

[25] C. L. Goues, T. Nguyen, S. Forrest, and W. Weimer, "Genprog: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, 2011.

[26] F. Long and M. Rinard, "Automatic patch generation by learning correct code," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT symposium on principles of programming languages (POPL)*, 2024.

[27] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2332–2354, 2024. [Online]. Available: https://doi.org/10.1145/3660810

[28] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "Specgen: Automated generation of formal program specifications via large language models," 2024. [Online]. Available: https://arxiv.org/abs/2401.08807

[29] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, "Can large language models transform natural language intent into formal method postconditions?" *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi-org.libproxy1.nus.edu.sg/10.1145/3660791

[30] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, "Llm-based test-driven interactive code generation: User study and empirical evaluation," *IEEE Trans. Software Eng.*, vol. 50, no. 9, pp. 2254–2268, 2024. [Online]. Available: https://doi.org/10.1109/TSE.2024.3428972

[31] T. Zhang, T. Yu, T. Hashimoto, M. Lewis, W. Yih, D. Fried, and S. Wang, "Coder reviewer reranking for code generation," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 2023, pp. 41 832–41 846. [Online]. Available: https://proceedings.mlr.press/v202/zhang23av.html

[32] M. Lyu, B. Ray, A. Roychoudhury, S. Tan, and P. Thongtanunam, "Automatic programming: Large language models and beyond," *arXiv:2405.02213*, 2024.

[33] AutoCodeRover, "Revamping issue resolution," Sep 2024, accessed: 2024-11-27. [Online]. Available: https://medium.com/@autocoderover/revamping-issue-resolution-f0d91ac6162c

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 11:42:33 UTC from IEEE Xplore. Restrictio